TA135
Taller de Automatización y Control

Facultad de Ingeniería
Universidad de Buenos Aires

Introduccion al Control Digital

Trabajo Práctico N°1

Grupo N°1

1º Cuatrimestre 2026

Autor:	**Padrón:**	**Correo:**
Dante Mele Ientile	109494	dmele@fi.uba.ar
Gonzalo Antahuara	109965	gantahuara@gmail.com

Índice

1. Limitaciones de los sensores **2**
1.1. Potenciómetro 2
1.2. Sensor de distancia ultrasónico (HC-SR04): 3
1.3. Unidad de Medición Inercial (IMU MPU6050) 4

2. Limitaciones de los actuadores **5**

3. Limitaciones de tareas adicionales **6**

4. Conclusiones **7**

1. Limitaciones de los sensores

Para el diseño del sistema de control de la planta, resulta fundamental identificar las limitaciones de los sensores encargados de la adquisición de datos. Conocer la resolución mínima de cada sensor es indispensable, permite determinar el error de cuantización del sistema, definiendo el cambio físico más pequeño que el sistema (Arduino UNO) será capaz de percibir para aplicar la acción de control.

Los sensores implementados en este trabajo práctico son:

- **Potenciómetro:** Podría emplearse como una entrada analógica para determinar el ángulo del servomotor.
- **Sensor de distancia ultrasónico (HC-SR04):** Funciona midiendo el tiempo de vuelo de un pulso sonoro de alta frecuencia que rebota contra una superficie. Se utilizará para medir de forma continua la posición lineal del carrito sobre la barra.
- **Unidad de Medición Inercial (IMU MPU6050):** Integra un acelerómetro de 3 ejes para medir la aceleración lineal y un giroscopio de 3 ejes que mide la velocidad angular. Se utilizará para estimar con precisión el ángulo de inclinación de la barra.

En las siguientes secciones se presentará un análisis detallado, basado en cálculos y estimaciones teóricas, para determinar la resolución mínima y las limitaciones de tiempo asociadas a cada sensor.[1]

1.1. Potenciómetro

El conversor analógico-digital (ADC) del Arduino posee una resolución de 10 bits, lo que le permite discretizar la señal analógica en $2^{10} = 1024$ niveles (de 0 hasta 1023). Dado que el potenciómetro se alimenta con 5V, la mínima variación de voltaje que el conversor es capaz de detectar se puede calcular relacionando el rango de tensión con los niveles discretizados:

$$R_V = \frac{5\,\mathrm{V}}{1024} \approx 4{,}89\,\mathrm{mV}$$

La resolución del sensor es de aproximadamente 4,89 mV. De esta manera, se define la mínima resolución que podemos obtener en la lectura digital obtenida mediante la función `analogRead()`.

En este orden de ideas, para calcular la mínima resolución angular del sensor, primero se obtuvo experimentalmente el recorrido mecánico máximo del potenciómetro, el cual es de 300°. Asumiendo un comportamiento idealmente lineal y mapeando el trayecto del potenciómetro a los 1024 niveles discretos del ADC, se obtiene:

$$R_{grados} = \frac{300^\circ}{1024} \approx 0{,}29^\circ$$

[1] El código fuente desarrollado para las validaciones empíricas de este informe se encuentra disponible en el siguiente repositorio: https://github.com/DanteMeleIentile/ta135-taller-control

En consecuencia, la resolución angular mínima que el controlador puede interpretar a partir de la entrada de referencia es de 0.29°. Por lo tanto, el sistema es incapaz de registrar variaciones de ángulos menores a este valor.

Para determinar el tiempo requerido al realizar una medición con el potenciómetro, es necesario analizar el proceso de conversión analógico-digital que realiza el microcontrolador. La duración de la lectura está condicionada por las especificaciones del hardware (frecuencia de reloj y configuración del ADC) y por el tiempo de ejecución del código en el microcontrolador.

- **Frecuencia del reloj del ADC:** El entorno de Arduino configura por defecto un preescalador de 128 para el conversor. Dado que el clock es de 16 MHz, el ADC opera a:

$$f_{ADC} = \frac{16000000}{128} = 125\,000\,\mathrm{Hz}$$

- **Ciclos de conversión:** Según la hoja de datos del microcontrolador, una conversión analógica ocupa aproximadamente 13 ciclos de su reloj. En consecuencia, el tiempo de conversión teórico resulta:

$$t_{conv} = \frac{13}{125000} = 104\,\mu\mathrm{s}$$

Para validar estos resultados experimentalmente, utilizamos el código `limitaciones_pot.ino`; allí se mide le tiempo promedio de 100 lecturas consecutivas mediante la función `micros()`, obteniendo un tiempo de medición real de 116,04 µs. La variación entre estos resultados se puede explicar considerando que la función `analogRead()` introduce instrucciones adicionales por software para configurar los registros y poner en marcha la lectura análogica.

1.2. Sensor de distancia ultrasónico (HC-SR04):

A diferencia de las señales analógicas, este sensor opera midiendo el Tiempo de Vuelo de un pulso sonoro a una frecuencia de 40 kHz. Por consiguiente, sus limitaciones provienen tanto de la gestión del tiempo del microcontrolador como de la física de las ondas acústicas.

La resolución espacial mínima del sensor ultrasónico, según hoja de datos del fabricante, es de 3 mm. Traducida al dominio del tiempo, esta cota impone al sensor una resolución temporal de ≈17,5 µs que resulta en un valor aceptable considerando la cuantización de 4 µs que presenta la función `micros()`.

A diferencia del potenciómetro, el tiempo de muestreo del sensor ultrasónico no es constante, sino que depende de la distancia al objetivo. Para evaluar el peor escenario de operación, se situó el carrito en el extremo de la barra (40 cm), registrando de forma empírica (código `limitaciones_sr04.ino`) un tiempo de respuesta de 2,82 ms.

Teóricamente, asumiendo una velocidad de propagación acústica de $340{,}29\,\mathrm{m\,s^{-1}}$ (en condiciones ambientales normales), el tiempo de vuelo del sonido para este trayecto resulta:

$$t_{vuelo} = \frac{2 \cdot 0{,}4\,\mathrm{m}}{340{,}29\,\mathrm{m\,s^{-1}}} \approx 2{,}35\,\mathrm{ms}$$

La diferencia de 0,47 ms entre el valor empírico y el teórico se puede atribuir, por un lado, a la latencia intrínseca del sensor (procesar trigger, excitar el transductor acústico y escrutar la

onda de eco recibida) y, por el otro, al retardo adicional introducido por el microcontrolador al ejecutar la función bloqueante de la función `sonar.ping()`.

1.3. Unidad de Medición Inercial (IMU MPU6050)

En estado estático, la velocidad angular ideal es nula, por lo que el giroscopio no aporta información de posición. En consecuencia, la estimación del ángulo se realiza con el acelerómetro del IMU mediante la relación trigonométrica `atan2(ay, az)`.

El sensor integra conversores ADC de 16 bits, que proporcionan $2^{16} = 65536$ niveles de cuantización discretos. Al configurar con un fondo de escala de $\pm 8g$ (mediante la instrucción `MPU6050_RANGE_8_G`), el transductor opera con una excursión total de $16g$. De este modo, la mínima variación de aceleración que el sistema puede digitalizar resulta:

$$\Delta a_{min} = \frac{16\text{g}}{65536} \approx 0{,}000244\text{g}$$

Asumiendo que el sistema se encuentra en reposo y en posición horizontal ($a_z \approx 1\text{g}$, $a_y \approx 0\text{g}$), la resolución angular se puede determinar mediante la propagación de la incertidumbre de cuantización sobre la función trigonométrica $\theta = \arctan\left(\frac{a_y}{a_z}\right)$ del siguiente modo:

$$\Delta\theta = \left|\frac{\partial\theta}{\partial a_y}\right| \Delta a_y + \left|\frac{\partial\theta}{\partial a_z}\right| \Delta a_z$$

Evaluando las derivadas parciales en el punto de operación de reposo:

$$\frac{\partial\theta}{\partial a_y} = \frac{a_z}{a_y^2 + a_z^2} \xrightarrow{a_y=0, a_z=1} 1$$

$$\frac{\partial\theta}{\partial a_z} = \frac{-a_y}{a_y^2 + a_z^2} \xrightarrow{a_y=0, a_z=1} 0$$

Reemplazando estos valores y considerando la incertidumbre mínima del ADC estimada anteriormente($\Delta a_y = \Delta a_z = \Delta a_{min}$), la resolución angular en radianes resulta:

$$\Delta\theta \approx (1 \cdot 0{,}000244) + (0 \cdot 0{,}000244) = 0{,}000244 \text{ rad}$$

Convirtiendo este valor a grados, se concluye que la mínima resolución teórica angular estática que nos ofrece la IMU es de **0.014°**.

Utilizando el código `limitaciones_imu.ino` se obtuvo un tiempo de medición de 3150 µs. Allí también se busca estimar el tiempo dedicado al invocar la función `mpu.getEvent()` (comunicación I2C y procesamiento interno de datos) y el tiempo estimado en el cálculo del ángulo mediante la función `atan2()`, obteniendo 3050 µs para el primero y 100 µs para el segundo.

Analíticamente, podemos hacer las siguientes consideraciones relacionadas con el proceso de adquisición de datos de la IMU:

- **Frecuencia de lectura:** Según la hoja de datos de fabricante, la frecuencia de operación para el acelerómetro es de 1 kHz y de 8 kHz para el giroscopio. Dado que nuestro control opera a 50 Hz (o 20 ms), este tiempo de conversión interno no genera un retraso en la lectura de datos.
- **Comunicación:** La función `mpu.getEvent()` se encarga de gestionar la comunicación I2C que consta de un paquete de 14 bytes de datos (6 bytes de aceleración, 6 de velocidad angular y 2 de temperatura). Considerando el overhead del protocolo, la trama transmitida es de 18 bytes (162 bits); por lo que, a la frecuencia de comunicación (100 kHz), el tiempo de transmisión teórico es:

$$t_{I2C} \approx \frac{162 \text{ bits}}{100\,000 \text{ Hz}} = 1620 \text{ µs}$$

En conclusión, estimamos que de los 3050 µs que lleva ejecutar la función `mpu.getEvent()`, se dedican unos 1620 µs a la comunicación I2C y los 1430 µs restantes en procesar los datos raw obtenidos.

El retardo asociado a la IMU, si bien es determinista y constante, consume cerca del 15 % del tiempo disponible en el ciclo principal del controlador configurado a 20 ms (50 Hz).

2. Limitaciones de los actuadores

El servomotor es comandado mediante una señal PWM a 50 Hz (período de 20 ms). Su posición angular depende exclusivamente de la duración del pulso en estado alto, cuyos límites de operacion se establecieron empíricamente entre 600 µs (0°) y 2400 µs (180°).

Para la generación de esta señal, la biblioteca `Servo.h` utiliza un timer 16 bits del microcontrolador. Al disponer de 65536 niveles de cuantización para el reloj principal del microcontrolador (16 MHz), es posible modular el ancho del pulso con una resolución temporal teórica de 1 µs.

Teóricamente, la precisión del servomotor queda condicionada por la función que se utilice para controlarlo:

- Al utilizar `write(angulo)`, la función procesa únicamente argumentos enteros. Esto introduce un truncamiento que acota la mínima variación angular a 1°.
- Al emplear `writeMicroseconds()`, se obtiene una resolución mayor. Considerando el ancho temporal útil de 1800 µs (2400 µs - 600 µs) requerido para efectuar un barrido de 180°, la resolución se calcula como:

$$\Delta\theta_{servo} = \frac{180°}{1800\ \mu\text{s}} = 0,1°/\mu\text{s}$$

Finalmente, si bien la precisión angular teórica es $\Delta\theta_{servo} = 0{,}1°$, empíricamente estimamos que no se detectan movimientos hasta introducir pulsos diferenciales de unos 16 µs, obteniendo $\Delta\theta_{servo} = 1{,}6°$

Es importante destacar que la mínima variación angular que experimentará la barra ($\Delta\theta_{barra}$) es proporcional a la resolución del servomotor pero está condicionada por la cinemática de la

planta, considerando la relación del mecanismo de transmisión entre el brazo del servo y el pivote de la barra.

Para determinar el tiempo que requiere el actuador en efectuar un desplazamiento diferencial de 30°, consideramos el valor indicado en la hoja de datos. Este modelo presenta una velocidad nominal de rotación de 0,17 s por cada 60° (operando a 4,8 V). Asumiendo un comportamiento de velocidad constante, el tiempo de respuesta del servo ante un incremento angular de $\Delta\theta = 30°$ resulta:

$$t_{\Delta 30°} = 30° \cdot \frac{0{,}17\,\mathrm{s}}{60°} = 0{,}085\,\mathrm{s} = 85\,\mathrm{ms}$$

Alternativamente, se puede estimar de forma empírica el tiempo de respuesta del actuador utilizando la IMU MPU6050. El procedimiento consiste armar la planta completa, colocar la IMU en la barra y comandar desde el microcontrolador un escalón de posición angular al servomotor y registrar mediante MATLAB el transitorio angular sensado por la IMU

No obstante, al analizar la curva de respuesta en MATLAB, se debe considerar que el tiempo medido no refleja el retardo real del actuador ya que el transitorio observado incluye los retardos de adquisición y procesamiento de la IMU y la latencia de la transmisión de datos a través del puerto serie.

Al utilizar una frecuencia de control de 50 Hz el tiempo requerido por el servo de 85 ms se puede representar como un retardo en el actuador. Dado que el lazo de control se actualiza cada 20 ms, el actuador demanda más de cuatro ciclos de muestreo para ejecutar la acción de control. Esta dinámica introduce características de fase no mínima al sistema que deben ser consideradas al diseñar el controlador.

Al bajar la frecuencia de control a 1 Hz tanto la precisión angular como el tiempo necesario para lograr un diferencial de 30° no se modifican pues los consideramos parámetros físicos intrínsecos del actuador que no dependen de la frecuencia del lazo de control.

No obstante, operar a una frecuencia de control de 1 Hz alteraría drásticamente el comportamiento pues el tiempo de respuesta físico del actuador resulta despreciable frente al retardo digital introducido por la baja frecuencia del lazo de control. Esta latencia obligaría a la acción de control a reaccionar ante estados pasados y obsoletos de la planta provocando, por ejemplo, acciones de control bruscas.

3. Limitaciones de tareas adicionales

Una variable de tipo `float` ocupa 4 bytes en memoria, lo que equivale a 32 bits de información. Sin embargo, la transmisión de estos datos se realiza mediante el periférico UART bajo el estándar 8N1. Esto implica que, por cada byte de datos, se agrega automáticamente un bit de inicio, ningún bit de paridad y un bit de parada, en consecuencia, enviar un único `float` requiere transmitir 40 bits. Operando a una velocidad de 115 200 bits/s (Baud Rate), el tiempo teórico de transmisión resulta:

$$t_{1_\mathrm{float}} = \frac{40 \text{ bits}}{115200 \text{ bits/s}} \approx 347{,}2\mu\mathrm{s} \implies t_{100_\mathrm{floats}} = 100 \cdot 347{,}2\mu\mathrm{s} = 34{,}72 \text{ ms}$$

Para estimar el tiempo de envío no podemos utilizar la estrategia de ubicar `Serial.write()` entre dos lecturas de `micros()`. Pues `Serial.write()` es una función no bloqueante que se limita a cargar los datos en un buffer de transmisión de 64 bytes y retornar al código instantáneamente, mientras que el puerto serie del microcontrolador transmite en segundo plano.

Con el código `envio_floats.ino` se buscó estimar el tiempo necesario para enviar 100 floats mediante el agregado de la función bloqueante `Serial.flush()`, que detiene la ejecución hasta liberar completamente el buffer de transmisión. Si se utilizara simplemente `Serial.write()`, no consideraríamos en la medición de tiempo los últimos 64 bytes del buffer, midiendo incorrectamente. El tiempo registrado de forma experimental fue de 34 012 µs.

4. Conclusiones

A partir del análisis individual de las limitaciones de hardware, se concluye que el sistema se encuentra temporalmente dominado por la Unidad de Medición Inercial (IMU). Con un tiempo de respuesta de 3,15 ms, su lectura consume aproximadamente el 15 % del tiempo disponible en el lazo de control (20 ms).

Desde el punto de vista del actuador, se introduce un retardo teórico de 85 ms (o > 4 muestras) para una acción de control de 30°, lo cual introduce dinámicas de fase no mínima en la planta.